

PONTIFÍCIA UNIVERSIDADE CATÓLICA DE GOIÁS ESCOLA POLITÉCNICA
E DE ARTES GRADUAÇÃO EM CIÊNCIA DA COMPUTAÇÃO



AED - COMPILADORES

Aluno: Rodrigo Fernandes Nogueira Vieira

GOIÂNIA, 2026

SUMÁRIO

1 Parte 1: Definição da Linguagem e Tokens	5
1.1 Descrição da Linguagem	5
1) Características Gerais	5
2) Exemplo de Programa Válido	5
1.2 Definição dos Tokens	6
1) Palavras Reservadas (Keywords)	6
2) Identificadores e Literais	7
3) Operadores	7
4) Delimitadores e Tokens Especiais	7
1.3 Regras Léxicas Gerais	8
1.4 Exemplo de Tokenização	8
1) Entrada Valida	8
2) Tokens Gerados	8
3) Entrada com Erro Léxico	9
4) Saida Esperada	9
2 Parte 2: Analisador Léxico (Lexer)	9
2.1 Visão Geral do Analisador Léxico	9
1) Características do Lexer	10
2) Arquitetura do Modulo	10
2.2 Funcionamento do Scanner	10
1) Reconhecimento de Identificadores e Keywords	11
2) Reconhecimento de Números	11
3) Operadores de Dois Caracteres	11
4) Tratamento de Strings	11
2.3 Recuperação de Erros Léxicos	12
1) Tipos de Erro Detectados	12
2) Formato da Mensagem de Erro	12
2.4 Exemplos de Execução	12
1) Entrada Valida (valido.pks)	12
2) Entrada com Erros Léxicos (erro_lexico.pks)	13
2.5 Estrutura de Arquivos do Projeto	14
3 Parte 3: Gramática e Analisador Sintático	14
3.1 Gramática Formal da Linguagem PokeScript	14

1) Produções da Gramática (BNF).....	14
3.2 Conjuntos FIRST e FOLLOW.....	15
1) Conjunto FIRST.....	15
2) Conjunto FOLLOW	16
3.3 Analisador Sintático (Parser)	17
1) Arquitetura	17
2) Precedência de Operadores	18
3) Tratamento de Erros Sintáticos.....	18
3.4 Arvore Sintática Abstrata (AST)	18
1) Nos da AST	19
2) Exemplo de AST Impressa.....	19
4 PARTE 4 — Execução / Interpretação	20
4.1 Visão Geral do Interpretador	20
4.2 Pipeline Completo de 3 Fases	20
4.3 Arquitetura do Interpretador	21
1) Classe Interpretador.....	21
2) Mecanismo de Despacho.....	21
4.4 Tabela de Símbolos (Symbol Table)	22
1) Declaração de Pokémon	22
2) Declaração de Variáveis Tipadas	22
4.5 Execução de Comandos	22
1) Condicional Se/Senão	23
2) Laço Enquanto/Faca com proteção de loop infinito.....	23
3) Bloco Batalha.....	23
4.6 Avaliação de Expressões	24
1) Hierarquia de Operadores.....	24
2) Concatenação de Texto	24
4.7 Sistema de Tipos e Coerção.....	25
1) Função _truthy()	25
2) Divisão com Resultado Exato	25
4.8 Tratamento de Erros de Execução.....	26
1) Tipos de Erros de Execução.....	26
2) Exemplo de Erro em Execução	26
4.9 Exemplo Completo com Saída do Interpretador	27
4.10 Resumo dos Arquivos da Parte 4.....	28

5 PARTE 5 — Tratamento de Erros e Organização do Projeto.....	28
5.1 Visão Geral do Tratamento de Erros.....	28
5.2 Erros Léxicos (Fase 1)	29
1) Tipos de Erros Léxicos	29
2) Implementação da Recuperação Léxica	29
3) Saida Real com 3 Erros Léxicos.....	29
5.3 Erros Sintáticos (Fase 2).....	30
1) Tipos de Erros Sintáticos	30
2) Implementação da Recuperação Sintática	30
3) Saida Real com 2 Erros Sintáticos.....	31
5.4 Erros de Execução (Fase 3)	31
1) Tipos de Erros de Execução.....	31
2) Implementacao da Classe ErroExecucao	31
3) Saida Real com Erro de Execução	32
5.5 Tabela Consolidada de Todos os Erros	32
5.6 Organização do Projeto	33
1) Módulos e Responsabilidades	33
2) Fluxo de Dados Entre os Módulos	34
3) Como Executar	34

1 Parte 1: Definição da Linguagem e Tokens

1.1 Descrição da Linguagem

PokeScript é uma linguagem de programação imperativa e interpretada, criada com temática do universo Pokémon. Seu objetivo é demonstrar os fundamentos de um compilador completo - análise léxica, sintática, semântica e interpretação - por meio de uma sintaxe lúdica e expressiva em português.

A linguagem permite declarar Pokémon com atributos personalizados, realizar batalhas com lógica condicional, manipular variáveis tipadas, executar laços de repetição e interagir com o usuário via entrada de dados.

1) Características Gerais

Característica	Valor
Paradigma	Imperativo / Interpretado
Tipagem	Estática e forte (inteiro, texto, pokémon)
Sensível a maiúsculas	Sim (dragonite e Dragonite são IDs distintos)
Comentários	Linha única com # (descartados pelo lexer)
Extensão de arquivo	.pks (PokeScript Source)
Estrutura de bloco	Abertura implícita, fechamento com a palavra fim

2) Exemplo de Programa Válido

O programa abaixo ilustra os principais recursos da linguagem:

```
# Declaração do Pokémon principal
pokemon dragonite
    nivel = 55
    hp = 180
    ataque = 134
fim

capturar nomeRival

enquanto dragonite.hp > 0 faca
    se dragonite.ataque > 100 e dragonite.nivel >= 50 entao
        dano = dragonite.ataque * 2
        mostrar "Dragonite usou Hiperrayo em " + nomeRival + "!"
        mostrar dano
        dragonite.hp = dragonite.hp - 10
    senao
        mostrar "Dragonite esta cansado..."
        dragonite.hp = 0
fim
```

```
fim  
mostrar "Batalha encerrada!"
```

1.2 Definição dos Tokens

Os tokens são as unidades lexicais mínimas da linguagem PokeScript. O analisador léxico percorre o código-fonte caractere a caractere, agrupando sequências e associando a cada token seu tipo, lexema, linha e coluna de ocorrência.

1) Palavras Reservadas (Keywords)

Palavras reservadas não podem ser usadas como identificadores. O lexer as reconhece com prioridade máxima sobre qualquer identificador.

Token	Lexema	Descrição
POKEMON	<code>pokemon</code>	Inicia a declaração de um Pokémon com seus atributos
FIM	<code>fim</code>	Fecha qualquer bloco aberto (pokémon, se, enquanto, batalha)
BATALHA	<code>batalha</code>	Inicia um bloco de batalha entre dois Pokémon
VS	<code>vs</code>	Separador entre os dois combatentes na batalha
SE	<code>se</code>	Início de estrutura condicional (if)
ENTAO	<code>entao</code>	Corpo do bloco verdadeiro da condicional (then)
SENAO	<code>senao</code>	Corpo do bloco alternativo da condicional (else)
ENQUANTO	<code>enquanto</code>	Início do laço de repetição (while)
FACA	<code>faca</code>	Delimita o corpo do laço enquanto...faca (do)
MOSTRAR	<code>mostrar</code>	Exibe um valor ou expressão no terminal (print)
CAPTURAR	<code>capturar</code>	Le entrada do usuário e armazena em variável (input)
INTEIRO	<code>inteiro</code>	Declara variável do tipo numérico inteiro
TEXTO	<code>texto</code>	Declara variável do tipo textual (string)
E	<code>e</code>	Operador logico AND (conjunção)
OU	<code>ou</code>	Operador logico OR (disjunção)
NAO	<code>nao</code>	Operador logico NOT (negação)
VERDADEIRO	<code>verdadeiro</code>	Literal booleano verdadeiro
FALSO	<code>falso</code>	Literal booleano falso

2) Identificadores e Literais

Token	Padrão (Regex)	Exemplos	Descrição
ID	<code>[a-zA-Z_][a-zA-Z0-9_]*</code>	<code>dragonite, hp, x1</code>	Nome de variável ou Pokémon
NUM_INT	<code>[0-9]+</code>	<code>55, 180, 0, 999</code>	Número inteiro
NUM_FLOAT	<code>[0-9]+[.][0-9]+</code>	<code>3.14, 99.5, 0.01</code>	Número de ponto flutuante
STRING	<code>"[^\\n]*"</code>	<code>"Hiperrayo!", "Lugia"</code>	Cadeia entre aspas duplas

3) Operadores

Token	Lexema	Descrição
ATRIB	<code>=</code>	Atribuição de valor a variável ou atributo
IGUAL	<code>==</code>	Comparação de igualdade entre dois valores
DIFER	<code>!=</code>	Comparação de diferença entre dois valores
MAIOR	<code>></code>	Comparação: maior que
MENOR	<code><</code>	Comparação: menor que
MAIOR_IG	<code>>=</code>	Comparação: maior ou igual a
MENOR_IG	<code><=</code>	Comparação: menor ou igual a
SOMA	<code>+</code>	Adição numérica ou concatenação de texto
SUB	<code>-</code>	Subtração numérica
MULT	<code>*</code>	Multiplicação numérica
DIV	<code>/</code>	Divisão numérica
PONTO	<code>.</code>	Acesso a atributo de Pokémon (dragonite.hp)

4) Delimitadores e Tokens Especiais

Token	Lexema	Descrição
ABRE_PAR	<code>(</code>	Abre parêntese - agrupa subexpressões
FECHA_PAR	<code>)</code>	Fecha parêntese - agrupa subexpressões

COMENTARIO	#.*	Comentário de linha - descartado, não gera token
EOF	-	Fim do arquivo - sentinela que encerra a análise

1.3 Regras Léxicas Gerais

Sensibilidade a maiúsculas: A linguagem diferencia maiúsculas de minúsculas. dragonite e Dragonite são identificadores distintos e não intercambiáveis.

Espaços e quebras de linha: Espaços, tabulações e quebras de linha são ignorados. Servem apenas como delimitadores naturais entre tokens, sem gerar token próprio.

Prioridade de Keywords: Palavras reservadas tem prioridade absoluta sobre identificadores. O lexer testa a lista de keywords antes de classificar qualquer sequência como ID.

Comentários: Todo texto após # até o final da linha é descartado integralmente. Comentários não geram token e não afetam a análise sintática.

Strings: Strings são delimitadas por aspas duplas e devem estar na mesma linha. Uma string não fechada gera erro léxico com indicação de linha e coluna.

Recuperação de erros: Ao encontrar um símbolo inválido, o lexer reporta o erro com linha e coluna, descarta o caractere e continua a tokenização. Isso permite listar todos os erros léxicos em uma única execução.

Números negativos: Não existem literais negativos na linguagem. Para negatividade, usa-se a expressão 0 - x ou (0 - valor).

1.4 Exemplo de Tokenização

1) Entrada Valida

```
# Declaracao do Pokemon principal
pokemon dragonite
    nivel = 55
    hp    = 180
fim
```

2) Tokens Gerados

Token	Lexema	Linha	Coluna	Observação
COMENTARIO	# Declaracao...	1	1	Descartado - não entra na análise sintática
POKEMON	pokemon	2	1	Palavra reservada

ID	dragonite	2	9	Nome do Pokémon (identificador)
ID	nivel	3	5	Atributo
ATRIB	=	3	11	Operador de atribuição
NUM_INT	55	3	13	Literal inteiro
ID	hp	4	5	Atributo
ATRIB	=	4	11	Operador de atribuição
NUM_INT	180	4	13	Literal inteiro
FIM	fim	5	1	Fecha bloco pokémon
EOF		6	1	Fim do arquivo

3) Entrada com Erro Léxico

```
x = 10 @ 2
```

4) Saída Esperada

```
Token(ID,      'x',  linha 1, coluna 1)
Token(ATRIB,   '=',  linha 1, coluna 3)
Token(NUM_INT, '10', linha 1, coluna 5)
[ERRO LEXICO] Linha 1, Coluna 8: simbolo '@' nao reconhecido.
Token(NUM_INT, '2',  linha 1, coluna 10)
Token(EOF,     '',   linha 1, coluna 11)
```

O analisador léxico não interrompe ao encontrar o erro. Ele registra o problema, descarta o símbolo inválido e continua a tokenização, permitindo que todos os erros léxicos sejam reportados em uma única execução.

2 Parte 2: Analisador Léxico (Lexer)

2.1 Visão Geral do Analisador Léxico

O analisador léxico (lexer) é a primeira fase de processamento do compilador PokeScript. Ele recebe o código-fonte como uma sequência de caracteres e o converte em uma sequência de tokens, que serão consumidos pelo analisador sintático na etapa seguinte.

O lexer da PokeScript foi implementado em Python de forma orientada a objetos, sem uso de bibliotecas externas de geração automática de scanners. Todo o reconhecimento de padrões é feito manualmente, percorrendo o código-fonte caractere a caractere.

1) Características do Lexer

Característica	Detalhe
Linguagem de implementação	Python 3.10+
Arquivo principal	lexer.py
Abordagem	Manual (sem geradores automáticos como PLY ou ANTLR)
Tratamento de erros	Recuperação: reporta e continua (não para no 1o erro)
Informações por token	Tipo, valor, linha e coluna de ocorrência
Comentários	Descartados automaticamente (não geram token)
Strings não fechadas	Detectadas e reportadas como erro léxico

2) Arquitetura do Modulo

O modulo lexer.py e organizado em quatro componentes principais:

Componente	Descrição
TipoToken (Enum)	Enumera todos os tipos de token reconhecidos pela linguagem
PALAVRAS_RESERVADAS (dict)	Mapeia lexemas de keywords para seus TipoToken correspondentes
Token (class)	Representa um token com tipo, valor, linha e coluna
Lexer (class)	Implementa o scanner com os métodos de tokenização

2.2 Funcionamento do Scanner

O método principal do lexer e tokenizar(). Ele executa um laco sobre o código-fonte e, a cada iteração, identifica o próximo token seguindo uma ordem de prioridade bem definida:

Prioridade	Padrão Verificado	Token Gerado
1	Espaço, tabulação, quebra de linha	Ignorado (sem token)
2	Caractere #	Comentário - descartado até fim da linha
3	Aspas duplas "	STRING - conteúdo até o próximo "
4	Digito [0-9]	NUM_INT ou NUM_FLOAT

5	Letra ou _ [a-zA-Z_]	Keyword ou ID
6	Dois caracteres: ==, !=, >=, <=	IGUAL, DIFER, MAIOR_IG, MENOR_IG
7	Um caractere: = > < + - * / . ()	Operador ou delimitador correspondente
8	Qualquer outro caractere	ERRO LEXICO - reporta e descarta

1) Reconhecimento de Identificadores e Keywords

Ao encontrar uma letra ou underscore, o lexer consome todos os caracteres alfanuméricos seguintes formando o lexema completo. Em seguida, consulta o dicionário PALAVRAS_RESERVADAS: se o lexema for uma keyword, gera o token correspondente; caso contrário, gera um token ID.

```
# Exemplo interno do metodo _ler_identificador()
valor = ""
while atual().isalnum() or atual() == "_":
    valor += avançar()
tipo = PALAVRAS_RESERVADAS.get(valor, TipoToken.ID)
return Token(tipo, valor, linha, coluna)
```

2) Reconhecimento de Números

O lexer primeiro consome todos os dígitos consecutivos. Se o próximo caractere for um ponto (.) seguido de mais dígitos, consome o ponto e a parte decimal, gerando NUM_FLOAT. Caso contrário, gera NUM_INT.

```
# Exemplos de reconhecimento
55      -> Token(NUM_INT, "55")
3.14    -> Token(NUM_FLOAT, "3.14")
180     -> Token(NUM_INT, "180")
```

3) Operadores de Dois Caracteres

Os operadores de dois caracteres (==, !=, >=, <=) são verificados antes dos operadores simples. O lexer espia o caractere seguinte (peek) sem consumi-lo, verificando se forma um par válido antes de avançar.

4) Tratamento de Strings

Ao encontrar aspas duplas, o lexer consome o conteúdo até encontrar o fechamento. Se chegar ao fim da linha sem fechar a string, registra um erro léxico e continua a análise a partir da próxima linha.

```
# String valida
"Dragonite usou Hiperrayo!" -> Token(STRING, "Dragonite usou
Hiperrayo!")

# String nao fechada
"resultado -> [ERRO LEXICO] string nao fechada
```

2.3 Recuperação de Erros Léxicos

Um diferencial importante do lexer da PokeScript e a recuperação de erros. Em vez de interromper a análise no primeiro erro encontrado, o scanner registra o problema em uma lista de erros, descarta o caractere invalido e continua processando o restante do arquivo.

Isso permite que o desenvolvedor visualize todos os erros léxicos de uma única vez, sem precisar corrigir um por um e reexecutar o programa a cada vez.

1) Tipos de Erro Detectados

Tipo de Erro	Exemplo	Mensagem Gerada
Símbolo invalido	x = 10 @ 2	símbolo '@' não reconhecido
String não fechada	mostrar "ola	string não fechada - falta '"' de fechamento
Caractere especial	y = x \$ 5	símbolo '\$' não reconhecido

2) Formato da Mensagem de Erro

Cada erro léxico e reportado com o padrão exigido pelo professor, indicando linha e coluna exatas:

```
[ERRO LEXICO] Linha 6, Coluna 8: simbolo '@' nao reconhecido
[ERRO LEXICO] Linha 7, Coluna 9: string nao fechada - falta '"' de
fechamento
[ERRO LEXICO] Linha 8, Coluna 7: simbolo '$' nao reconhecido
```

2.4 Exemplos de Execução

O lexer e executado via linha de comando, recebendo um arquivo .pks como argumento:

```
python main.py examples/valido.pks
python main.py examples/erro_lexico.pks
```

1) Entrada Valida (valido.pks)

Trecho da entrada:

```
# Declaracao do Pokemon principal
pokemon dragonite
    nivel = 55
    hp    = 180
    ataque = 134
fim
```

Saida gerada pelo lexer:

```
Token(POKEMON      "pokemon"   linha 4, col 1)
Token(ID           "dragonite"  linha 4, col 9)
Token(ID           "nivel"     linha 5, col 5)
Token(ATRIB       "="         linha 5, col 12)
Token(NUM_INT     "55"        linha 5, col 14)
Token(ID          "hp"        linha 6, col 5)
Token(ATRIB       "="         linha 6, col 12)
Token(NUM_INT     "180"       linha 6, col 14)
Token(ID          "ataque"     linha 7, col 5)
Token(ATRIB       "="         linha 7, col 12)
Token(NUM_INT     "134"       linha 7, col 14)
Token(FIM         "fim"       linha 8, col 1)

Analise lexica concluida sem erros.
Total de tokens : 71 | Total de erros : 0
```

2) Entrada com Erros Léxicos (erro_lexico.pks)

Entrada:

```
x = 10 @ 2
mostrar "resultado
y = x $ 5
```

Saida gerada pelo lexer:

```
Token(ID,         "x",   linha 6, col 1)
Token(ATRIB,      "=",   linha 6, col 3)
Token(NUM_INT,    "10",  linha 6, col 5)
Token(NUM_INT,    "2",   linha 6, col 10)
Token(MOSTRAR,    "mostrar", linha 7, col 1)
...

ERROS LEXICOS ENCONTRADOS (3):
[ERRO LEXICO] Linha 6, Coluna 8: simbolo '@' nao reconhecido
[ERRO LEXICO] Linha 7, Coluna 9: string nao fechada
[ERRO LEXICO] Linha 8, Coluna 7: simbolo '$' nao reconhecido

Total de tokens : 10 | Total de erros : 3
```

2.5 Estrutura de Arquivos do Projeto

Arquivo	Função
<code>lexer.py</code>	Analizador léxico completo (TipoToken, Token, ErroLexico, Lexer)
<code>main.py</code>	Ponto de entrada - recebe arquivo .pks e exibe tokens e erros
<code>examples/valido.pks</code>	Programa PokeScript valido para demonstração
<code>examples/erro_lexico.pks</code>	Programa com erros léxicos para demonstração de recuperação

3 Parte 3: Gramática e Analisador Sintático

3.1 Gramática Formal da Linguagem PokeScript

A gramática da PokeScript e uma Gramática Livre de Contexto (GLC) na forma BNF (Backus-Naur Form), projetada para ser compatível com análise preditiva LL(1). As principais propriedades garantidas são: ausência de recursão a esquerda e ausência de ambiguidade nas produções.

A hierarquia de precedência dos operadores e codificada diretamente na estrutura da gramática, do nível mais baixo (ou logico) ao mais alto (fator/literal).

1) Produções da Gramática (BNF)

```
programa      -> declaracao* EOF

declaracao    -> decl_pokemon
               | decl_batalha
               | comando

decl_pokemon  -> POKEMON ID atrib_pok* FIM

atrib_pok     -> ID ATRIB expressao

decl_batalha  -> BATALHA ID VS ID bloco FIM

bloco         -> comando*

comando       -> MOSTRAR expressao
               | CAPTURAR ID
               | SE expressao ENTAO bloco (SENAO bloco)? FIM
               | ENQUANTO expressao FACA bloco FIM
               | INTEIRO ID ATRIB expressao
               | TEXTO   ID ATRIB expressao
               | ID ATRIB expressao
               | ID PONTO ID ATRIB expressao
```

```

expressao      -> expr_ou

expr_ou        -> expr_e (OU expr_e)*

expr_e         -> expr_nao (E expr_nao)*

expr_nao       -> NAO expr_nao
                | expr_comp

expr_comp      -> expr_soma (op_comp expr_soma)?

op_comp        -> IGUAL | DIFER | MAIOR | MENOR | MAIOR_IG |
MENOR_IG

expr_soma      -> expr_mult ((SOMA | SUB) expr_mult)*

expr_mult      -> expr_unario ((MULT | DIV) expr_unario)*

expr_unario    -> SUB expr_unario
                | fator

fator          -> NUM_INT
                | NUM_FLOAT
                | STRING
                | VERDADEIRO
                | FALSO
                | ABRE_PAR expressao FECHA_PAR
                | ID
                | ID PONTO ID

```

3.2 Conjuntos FIRST e FOLLOW

Os conjuntos FIRST e FOLLOW são calculados para cada não-terminal da gramática e são a base matemática que garante que o parser preditivo LL(1) pode decidir, com apenas um token de lookahead, qual produção aplicar em cada passo da análise.

1) Conjunto FIRST

$FIRST(A)$ = conjunto de terminais que podem aparecer como primeiro símbolo em qualquer forma sentencial derivada de A.

Não-Terminal	FIRST(X) — primeiros terminais que podem iniciar uma derivação
programa	POKEMON, BATALHA, MOSTRAR, CAPTURAR, SE, ENQUANTO, INTEIRO, TEXTO, ID, EOF
declaracao	POKEMON, BATALHA, MOSTRAR, CAPTURAR, SE, ENQUANTO, INTEIRO, TEXTO, ID

decl_pokemon	POKEMON
atrib_pok	ID
decl_batalha	BATALHA
bloco	MOSTRAR, CAPTURAR, SE, ENQUANTO, INTEIRO, TEXTO, ID, e (vazio)
comando	MOSTRAR, CAPTURAR, SE, ENQUANTO, INTEIRO, TEXTO, ID
expressao	NUM_INT, NUM_FLOAT, STRING, VERDADEIRO, FALSO, ABRE_PAR, ID, NAO, SUB
expr_ou	NUM_INT, NUM_FLOAT, STRING, VERDADEIRO, FALSO, ABRE_PAR, ID, NAO, SUB
expr_e	NUM_INT, NUM_FLOAT, STRING, VERDADEIRO, FALSO, ABRE_PAR, ID, NAO, SUB
expr_nao	NAO, NUM_INT, NUM_FLOAT, STRING, VERDADEIRO, FALSO, ABRE_PAR, ID, SUB
expr_comp	NUM_INT, NUM_FLOAT, STRING, VERDADEIRO, FALSO, ABRE_PAR, ID, NAO, SUB
expr_soma	NUM_INT, NUM_FLOAT, STRING, VERDADEIRO, FALSO, ABRE_PAR, ID, NAO, SUB
expr_mult	NUM_INT, NUM_FLOAT, STRING, VERDADEIRO, FALSO, ABRE_PAR, ID, NAO, SUB
expr_unario	SUB, NUM_INT, NUM_FLOAT, STRING, VERDADEIRO, FALSO, ABRE_PAR, ID
fator	NUM_INT, NUM_FLOAT, STRING, VERDADEIRO, FALSO, ABRE_PAR, ID

2) Conjunto FOLLOW

FOLLOW(A) = conjunto de terminais que podem aparecer imediatamente após A em alguma forma sentencial. Usado para decidir quando aplicar uma produção que deriva a palavra vazia.

Não-Terminal	FOLLOW(X) — terminais que podem seguir o nao-terminal
programa	EOF (\$)
declaracao	POKEMON, BATALHA, MOSTRAR, CAPTURAR, SE, ENQUANTO, INTEIRO, TEXTO, ID, EOF
decl_pokemon	POKEMON, BATALHA, MOSTRAR, CAPTURAR, SE, ENQUANTO, INTEIRO, TEXTO, ID, EOF
atrib_pok	FIM, ID

decl_batalha	POKEMON, BATALHA, MOSTRAR, CAPTURAR, SE, ENQUANTO, INTEIRO, TEXTO, ID, EOF
bloco	FIM, SENAO
comando	MOSTRAR, CAPTURAR, SE, ENQUANTO, INTEIRO, TEXTO, ID, FIM, SENAO
expressao	ENTAO, FACA, FIM, SENAO, MOSTRAR, CAPTURAR, SE, ENQUANTO, INTEIRO, TEXTO, ID, EOF
expr_ou	ENTAO, FACA, FIM, SENAO, MOSTRAR, CAPTURAR, SE, ENQUANTO, INTEIRO, TEXTO, ID, EOF
expr_e	OU, ENTAO, FACA, FIM, SENAO, MOSTRAR, CAPTURAR, SE, ENQUANTO, INTEIRO, TEXTO, ID, EOF
expr_nao	E, OU, ENTAO, FACA, FIM, SENAO, MOSTRAR, CAPTURAR, SE, ENQUANTO, INTEIRO, TEXTO, ID, EOF
expr_comp	E, OU, ENTAO, FACA, FIM, SENAO, MOSTRAR, CAPTURAR, SE, ENQUANTO, INTEIRO, TEXTO, ID, EOF
expr_soma	IGUAL, DIFER, MAIOR, MENOR, MAIOR_IG, MENOR_IG, E, OU, ENTAO, FACA, FIM, SENAO, EOF
expr_mult	SOMA, SUB, IGUAL, DIFER, MAIOR, MENOR, MAIOR_IG, MENOR_IG, E, OU, ENTAO, FACA, FIM, SENAO, EOF
expr_unario	MULT, DIV, SOMA, SUB, IGUAL, DIFER, MAIOR, MENOR, MAIOR_IG, MENOR_IG, E, OU, ENTAO, FACA, FIM, SENAO, EOF
fator	MULT, DIV, SOMA, SUB, IGUAL, DIFER, MAIOR, MENOR, MAIOR_IG, MENOR_IG, E, OU, ENTAO, FACA, FIM, SENAO, EOF

3.3 Analisador Sintático (Parser)

O parser da PokeScript é um analisador recursivo descendente, onde cada não-terminal da gramática corresponde diretamente a um método Python. A análise é preditiva (LL(1)): a cada passo, o parser consulta apenas o token atual para decidir qual produção aplicar, sem necessidade de retrocesso (backtracking).

Além de validar a estrutura sintática, o parser constrói simultaneamente a Árvore Sintática Abstrata (AST), que representa a hierarquia lógica do programa e serve de entrada para o interpretador.

1) Arquitetura

Arquivo	Conteúdo
<code>parser.py</code>	Classe Parser com um método por não-terminal da gramática
<code>ast_nodes.py</code>	Classes dos nós da AST (NodoPokemon, NodoSe, NodoBinario, etc.)

`main.py`

Integra lexer e parser, imprime tokens, AST e relatório de erros

2) Precedência de Operadores

A precedência é codificada na hierarquia de chamadas dos métodos de expressão, do nível mais baixo ao mais alto:

Nível	Método	Operadores	Associatividade
1 (mais baixo)	<code>_expr_ou</code>	ou	Esquerda
2	<code>_expr_e</code>	e	Esquerda
3	<code>_expr_nao</code>	nao	Direita (prefixo)
4	<code>_expr_comp</code>	<code>== != > < >= <=</code>	Nao-associativo
5	<code>_expr_soma</code>	<code>+ -</code>	Esquerda
6	<code>_expr_mult</code>	<code>* /</code>	Esquerda
7	<code>_expr_unario</code>	<code>-</code> (unario)	Direita (prefixo)
8 (mais alto)	<code>_fator</code>	NUM, STRING, ID, ID.attr, ()	N/A

3) Tratamento de Erros Sintáticos

O parser implementa recuperação básica de erros: ao encontrar um token inesperado, registra o erro (com linha e coluna) e tenta continuar a análise. Isso permite que múltiplos erros sintáticos sejam reportados em uma única execução.

```
# Exemplo de saída com erro sintatico

[ERRO SINTATICO] Linha 9, Coluna 1: esperado "FIM", encontrado "se"
(SE)
[ERRO SINTATICO] Linha 10, Coluna 5: esperado "ENTAO", encontrado
"mostrar" (MOSTRAR)

Status: REJEITADO
```

3.4 Arvore Sintática Abstrata (AST)

A AST é a representação hierárquica do programa após a análise sintática. Cada nó representa uma construção semântica: declaração de Pokémon, comando, expressão, etc. O `impressorAST` percorre a árvore e a exibe indentada no terminal.

1) Nos da AST

Classe	Representa
NodoPrograma	Raiz da arvore — lista de todas as declarações do programa
NodoPokemon	Declaração pokemon nome ... fim
NodoAtribPokemon	Atributo dentro de um bloco pokemon
NodoBatalha	Declaração batalha nome1 vs nome2 ... fim
NodoDeclVar	Declaração de variavel tipada: inteiro x = 10
NodoAtrib	Atribuicao: x = expr ou dragonite.hp = expr
NodoMostrar	Comando mostrar expressão
NodoCapturar	Comando capturar variável (leitura)
NodoSe	Condicional se ... entao ... (senao ...) fim
NodoEnquanto	Laco enquanto ... faca ... fim
NodoBinario	Operação binaria: +, -, *, /, ==, >, e, ou, etc.
NodoUnario	Operação unária: - (negativo) ou não (negação logica)
NodoNumero	Literal numérico inteiro ou flutuante
NodoString	Literal textual entre aspas duplas
NodoBooleano	Literal verdadeiro ou falso
NodoIdentificador	Nome de variável simples
NodoAcesso	Acesso a atributo de Pokémon: dragonite.hp

2) Exemplo de AST Impressa

Para a entrada valido.pks, o compilador produz a seguinte AST:

```
Programa
  Pokemon: dragonite (linha 4)
    Atributo: nivel =
      Numero: 55
    Atributo: hp =
      Numero: 180
  Capturar: nomeRival (linha 10)
  Enquanto: (linha 12)
    Condicao:
      BinOp: '>'
      Acesso: dragonite.hp
      Numero: 0
  Corpo:
    Se: (linha 13)
      Condicao:
```

```

        BinOp: 'e'
        BinOp: '>'
        Acesso: dragonite.ataque
        Numero: 100
        BinOp: '>='
        Acesso: dragonite.nivel
        Numero: 50
Entao:
  Atrib: dano = dragonite.ataque * 2
  Mostrar: "Dragonite usou Hiperrayo em " + ...
Senao:
  Mostrar: "Dragonite esta cansado..."
Mostrar: "Batalha encerrada!"

```

4 PARTE 4 — Execução / Interpretação

4.1 Visão Geral do Interpretador

A Parte 4 do Mini-Compilador PokeScript implementa a fase de execução por meio de um interpretador tree-walker (ou tree-walker interpreter). Nessa abordagem, a AST construída pelo parser é percorrida diretamente, sem geração de código intermediário ou bytecode. Cada nó da árvore possui um método de execução correspondente que realiza a operação semântica imediatamente.

O interpretador está implementado no módulo `interpreter.py` e invocado pelo `main.py` após as fases léxica e sintática concluírem sem erros críticos.

4.2 Pipeline Completo de 3 Fases

O arquivo `main.py` orquestra as três fases em sequência. O programa só executa se as fases anteriores forem bem-sucedidas:

```

$ python main.py examples/batalha.pks

=====
PokeScript -- examples/batalha.pks
=====

[ FASE 1 ]  Analise Lexica
OK  --  198 tokens gerados

[ FASE 2 ]  Analise Sintatica
OK  --  AST construida

[ FASE 3 ]  Execucao
...
Status      : ACEITO E EXECUTADO

```

Fase	Modulo	Entrada	Saida
Fase 1 — Léxica	<code>lexer.py</code>	Codigo-fonte .pks	Lista de Tokens + erros lexicos
Fase 2 — Sintática	<code>parser.py</code>	Lista de Tokens	AST (NodoPrograma) + erros sint.
Fase 3 — Execução	<code>interpreter.py</code>	NodoPrograma (AST)	Saida no terminal + tabela vars

4.3 Arquitetura do Interpretador

O interpretador tree-walker e composto por uma única classe Interpretador que: (1) mantém o estado do programa (variáveis e Pokémon); (2) percorre recursivamente a AST; (3) executa cada no chamando o método adequado via um dicionário de despacho.

1) Classe Interpretador

```
class Interpretador:
    ambiente: dict # variaveis simples -> valor
    pokemons: dict # pokemon -> { atributo -> valor }
    erros: list # ErroExecucao coletados

    def executar(self, programa: NodoPrograma)
    def _exec(self, nodo) # despacho por tipo de no
    def _avaliar(self, nodo) # avaliacao de expressoes
```

2) Mecanismo de Despacho

O método `_exec()` utiliza um dicionário que mapeia cada tipo de no AST para o método de execução correspondente:

```
def _exec(self, nodo):
    metodo = {
        NodoPokemon: self._exec_pokemon,
        NodoBatalha: self._exec_batalha,
        NodoDeclVar: self._exec_decl_var,
        NodoAtrib: self._exec_atrib,
        NodoMostrar: self._exec_mostrar,
        NodoCapturar: self._exec_capturar,
        NodoSe: self._exec_se,
        NodoEnquanto: self._exec_enquanto,
    }.get(type(nodo))
    if metodo: metodo(nodo)
```

4.4 Tabela de Símbolos (Symbol Table)

A tabela de símbolos é implementada como dois dicionários Python separados, mantidos em memória durante toda a execução do programa:

Dicionário	Chave	Valor	Uso
<code>ambiente</code>	nome da variável	valor Python (int/float/str/bool)	Variáveis declaradas com inteiro/texto ou por atribuição
<code>pokemons</code>	nome do Pokémon	dict { atrib -> valor }	Entidades Pokémon com seus atributos

Quando um Pokémon é declarado, ele é registrado tanto em `pokemons` quanto em `ambiente` (permitindo usar o nome do Pokémon como variável). As alterações de atributos são sincronizadas entre os dois dicionários automaticamente.

1) Declaração de Pokémon

```
# Código PokeScript:
pokemon dragonite
    nivel = 55
    hp    = 250
    ataque = 134
fim

# Estado interno após execução:
pokemons = {
    'dragonite': { 'nivel': 55, 'hp': 250, 'ataque': 134 }
}
ambiente = {
    'dragonite': { 'nivel': 55, 'hp': 250, 'ataque': 134 } # ref
}
```

2) Declaração de Variáveis Tipadas

```
# Código PokeScript:
inteiro dano = dragonite.ataque * 2
texto msg    = "Ataque poderoso!"

# Estado interno:
ambiente = { 'dano': 268, 'msg': 'Ataque poderoso!' }
```

4.5 Execução de Comandos

Comando PokeScript	Método Interpretador	Comportamento
--------------------	----------------------	---------------

mostrar expr	_exec_mostrar	Avalia expr e imprime com print()
capturar id	_exec_capturar	Le input(), infere tipo (int/float/str)
se cond entao ... fim	_exec_se	Avalia condição, executa bloco adequado
enquanto cond faca ...fim	_exec_enquanto	Loop com limite de 10.000 iterações
inteiro id = expr	_exec_decl_var	Cria variável com coerção de tipo
id = expr	_exec_atrib	Atualiza variável no ambiente
id.attr = expr	_exec_atrib	Atualiza atributo de Pokémon
pokemon id ... fim	_exec_pokemon	Registra Pokémon e atributos
batalha p1 vs p2 faca...fim	_exec_batalha	Exibe separador visual e executa bloco

1) Condicional Se/Senao

```
se dragonite.hp > 100 entao
    mostrar "Dragonite esta forte!"
senao
    mostrar "Dragonite precisa descansar."
fim
```

2) Laco Enquanto/Faca com proteção de loop infinito

```
# O interpretador aborta apos 10.000 iteracoes:
enquanto contador < 5 faca
    mostrar contador
    contador = contador + 1
fim

# Se o limite for excedido:
# [ERRO EXECUCAO]: limite de iteracoes atingido (possivel loop infinito)
```

3) Bloco Batalha

```
batalha dragonite vs charizard faca
    dano = dragonite.ataque - charizard.defesa
    charizard.hp = charizard.hp - dano
    mostrar "Dragonite causou " + dano + " de dano!"
fim

# Saida no terminal:
-----
>> BATALHA: dragonite vs charizard <<
-----
Dragonite causou 54 de dano!
```

4.6 Avaliação de Expressões

O método `_avaliar()` é responsável por calcular o valor de qualquer expressão. Ele distingue os tipos de nós por `isinstance()` e retorna um valor Python nativo.

Tipo de Expressão	No AST	Exemplo PokeScript	Valor Retornado
Número inteiro	<code>NodoNumero</code>	<code>42</code>	<code>int 42</code>
Número real	<code>NodoNumero</code>	<code>3.14</code>	<code>float 3.14</code>
Texto literal	<code>NodoString</code>	<code>"Pika!"</code>	<code>str 'Pika!'</code>
Booleano	<code>NodoBooleano</code>	<code>verdadeiro</code>	<code>bool True</code>
Identificador	<code>NodoIdentificador</code>	<code>hp</code>	valor em ambiente
Acesso a atributo	<code>NodoAcesso</code>	<code>dragonite.hp</code>	<code>pokemons['dragonite']['hp']</code>
Operação binária	<code>NodoBinario</code>	<code>ataque * 2</code>	resultado da operação
Operação unária	<code>NodoUnario</code>	<code>nao verdadeiro</code>	<code>bool False</code>

1) Hierarquia de Operadores

A precedência é resolvida estruturalmente pela AST (determinada pelo parser). No interpretador, os operadores são avaliados de acordo com a ordem dos nós:

Nível	Operadores	Associatividade	Exemplo
1 (menor)	<code>ou</code>	Esquerda	<code>a ou b</code>
2	<code>e</code>	Esquerda	<code>a e b</code>
3	<code>nao</code>	Direita	<code>nao verdadeiro</code>
4	<code>== != > < >= <=</code>	Nao-assoc	<code>a > b</code>
5	<code>+ -</code>	Esquerda	<code>a + b</code>
6	<code>* /</code>	Esquerda	<code>a * b</code>
7	<code>- (unario)</code>	Direita	<code>-x</code>
8 (maior)	<code>numero, texto, id, id.id, ()</code>	--	<code>dragonite.hp</code>

2) Concatenação de Texto

O operador `+` realiza concatenação automaticamente se qualquer operando for texto. Isso permite montar mensagens misturando variáveis numéricas e strings:

```
mostrar "Dragonite causou " + dano + " de dano!"
```



```
# dano = 268 (inteiro)
# Resultado: "Dragonite causou 268 de dano!"
```

3) Curto-Circuito Logico

Os operadores `e` ou implementam curto-circuito (short-circuit evaluation), evitando avaliar o segundo operando quando o resultado já é determinado pelo primeiro:

```
# 'e' para se esquerda for falso:
se hp > 0 e ataque > 100 entao # 'ataque > 100' so e avaliado se
'hp > 0' for verdadeiro
...
fim

# 'ou' para se esquerda for verdadeiro:
se nivel >= 50 ou hp > 200 entao # 'hp > 200' so e avaliado se
'nivel >= 50' for falso
...
fim
```

4.7 Sistema de Tipos e Coerção

PokeScript é uma linguagem de tipagem dinâmica — os tipos são verificados em tempo de execução. Quando uma variável é declarada com inteiro ou texto, o interpretador aplica coerção:

Declaracao	Comportamento	Exemplo
<code>inteiro x = expr</code>	Converte valor para int via <code>int()</code> . Erro se não conversível.	<code>`inteiro dano = 134 * 2` -> dano = 268</code>
<code>texto x = expr</code>	Converte para string via <code>_para_texto()</code> .	<code>`texto hp_str = dragonite.hp` -> '250'</code>
<code>x = expr</code>	Sem declaração de tipo: valor inferido automaticamente.	<code>`resultado = 10 / 4` -> 2 (divisão exata)</code>

1) Função `_truthy()`

A função `_truthy()` converte qualquer valor para booleano, permitindo usar números e textos em condições (assim como Python):

```
# Verdadeiro:
0 -> falso | qualquer outro numero -> verdadeiro
"" -> falso | qualquer string não-vazia -> verdadeiro
falso -> falso | verdadeiro -> verdadeiro
```

2) Divisão com Resultado Exato

```
resultado = 10 / 2    # -> 5   (inteiro, divisao exata)
resultado = 10 / 3    # -> 3.333... (float, divisao inexata)
```

4.8 Tratamento de Erros de Execução

Erros semânticos em tempo de execução são representados pela classe `ErroExecucao`. Quando um erro ocorre, a execução é interrompida e a mensagem é exibida com contexto de linha:

```
class ErroExecucao(Exception):
    def __init__(self, mensagem, linha=None)

# Representacao:
[ERRO EXECUCAO] (linha 12): variavel 'hp' nao foi declarada
```

1) Tipos de Erros de Execução

Tipo de Erro	Mensagem	Causa
Variável não declarada	<code>variavel 'x' nao foi declarada</code>	Uso de variável antes de atribuição
Pokémon não declarado	<code>Pokemon 'X' nao declarado para batalha</code>	Bloco batalha referência Pokémon inexistente
Atributo inexistente	<code>atributo 'X' nao encontrado em 'P'</code>	Acesso a atributo não definido no Pokémon
Divisão por zero	<code>divisao por zero</code>	Operação de divisão com divisor = 0
Tipo invalido	<code>nao e possivel converter 'X' para inteiro</code>	Coerção de tipo incompatível
Loop infinito	<code>limite de iteracoes atingido</code>	Laco enquanto atingiu 10.000 iterações
No desconhecido	<code>no desconhecido: NomeDoNode</code>	Tipo de no não mapeado no interpretador

2) Exemplo de Erro em Execução

```
# Programa PokeScript com erro:
mostrar resultado

# Saida (resultado nao foi declarado):
[ FASE 3 ] Execucao
-----
[ERRO EXECUCAO]: variavel 'resultado' nao foi declarada
```

4.9 Exemplo Completo com Saida do Interpretador

O programa abaixo demonstra todas as funcionalidades do interpretador em um único arquivo. Entrada do usuário: Rodrigo

Código-fonte (batalha.pks)

```
pokemon dragonite
    nivel  = 55
    hp     = 250
    ataque = 134
    defesa = 90
fim

pokemon charizard
    nivel  = 52
    hp     = 210
    ataque = 122
    defesa = 80
fim

mostrar "Treinador, qual o seu nome?"
capturar treinador

batalha dragonite vs charizard faca
    dano_drag = dragonite.ataque - charizard.defesa
    charizard.hp = charizard.hp - dano_drag
    mostrar "Dragonite causou " + dano_drag + " de dano!"
    dano_char = charizard.ataque - dragonite.defesa
    dragonite.hp = dragonite.hp - dano_char
    mostrar "Charizard causou " + dano_char + " de dano!"
fim

mostrar "HP Dragonite: " + dragonite.hp
mostrar "HP Charizard: " + charizard.hp
```

Saida no Terminal

```
Treinador, qual o seu nome?
Entrada (treinador): Rodrigo

-----
>> BATALHA: dragonite vs charizard <<
-----
Dragonite causou 54 de dano!
Charizard causou 32 de dano!
-----

HP Dragonite: 218
HP Charizard: 156
```

```
Estado final das variaveis:
```

```
-----  
treinador      = Rodrigo  
dano_drag      = 54  
dano_char      = 32  
[pokemon] dragonite:  
  nivel        = 55  
  hp           = 218  
  ataque       = 134  
  defesa       = 90  
[pokemon] charizard:  
  nivel        = 52  
  hp           = 156  
  ataque       = 122  
  defesa       = 80  
  
=====
```

```
Tokens      : 198  
Erros lex   : 0  
Erros sin   : 0  
Status      : ACEITO E EXECUTADO  
=====
```

4.10 Resumo dos Arquivos da Parte 4

Arquivo	Função
<code>interpreter.py</code>	Classe Interpretador + ErroExecucao — 100% da logica de execução
<code>main.py</code>	Orquestra as 3 fases: Lexer -> Parser -> Interpretador
<code>examples/batalha.pks</code>	Programa de demonstração completo com batalha Pokémon
<code>examples/valido.pks</code>	Programa simples demonstrando laço e condicional
<code>examples/erro_lexico.pks</code>	Demonstra recuperação de erros léxicos (3 erros)
<code>examples/erro_sintatico.pks</code>	Demonstra recuperação de erros sintáticos (2 erros)

5 PARTE 5 — Tratamento de Erros e Organização do Projeto

5.1 Visão Geral do Tratamento de Erros

O Mini-Compilador PokeScript implementa tratamento e recuperação de erros nas três fases do pipeline: análise léxica, análise sintática e interpretação. O objetivo é detectar o maior número possível de erros em uma única execução, continuando a análise após cada falha.

Cada fase possui sua própria classe de erro com informações de localização (linha e coluna), permitindo ao usuário identificar e corrigir problemas com precisão.

Fase	Classe de Erro	Informações	Comportamento
------	----------------	-------------	---------------

Lexica	ErroLexico	Linha, Coluna, Mensagem	Ignora o caractere invalido e continua tokenizando
Sintatica	ErroSintatico	Linha, Coluna, Mensagem	Registra o erro e tenta sincronizar no próximo token valido
Execucao	ErroExecucao	Linha, Mensagem	Lança exceção Python, interrompe a execução e exhibe contexto

5.2 Erros Léxicos (Fase 1)

A fase léxica detecta erros de caracteres inválidos e literais mal-formados. A recuperação é feita por avanço simples: o caractere invalido é registrado como erro e o lexer continua a partir do próximo caractere, sem interromper a tokenização.

1) Tipos de Erros Léxicos

Tipo de Erro	Exemplo no Código	Mensagem Gerada
Caractere nao reconhecido	<code>x = 10 @ 2</code>	[ERRO LEXICO] Linha 6, Coluna 8: simbolo '@' nao reconhecido
String nao fechada	<code>mostrar "texto</code>	[ERRO LEXICO] Linha 7, Coluna 9: string nao fechada -- falta '"' de fechamento
Simbolo invalido	<code>y = x \$ 5</code>	[ERRO LEXICO] Linha 8, Coluna 7: simbolo '\$' nao reconhecido

2) Implementação da Recuperação Léxica

```
# Em lexer.py -- ao encontrar caractere nao reconhecido:
else:
    self.erros.append(ErroLexico(
        f"simbolo '{self.atual()}' nao reconhecido",
        self.linha, self.coluna
    ))
    self._avancar() # ignora o caractere e continua
```

3) Saida Real com 3 Erros Léxicos

```
$ python main.py examples/erro_lexico.pks

[ FASE 1 ]  Analise Lexica
-----
Token(ID      "x"    linha 6, col 1)
Token(ATRIB   "="    linha 6, col 3)
Token(NUM_INT "10"   linha 6, col 5)
Token(NUM_INT "2"    linha 6, col 10)
Token(MOSTRAR "mostrar" linha 7, col 1)
```

```
Token(ID          "y"      linha 8, col 1)
...

Erros lexicos encontrados (3):
[ERRO LEXICO] Linha 6, Coluna 8: simbolo '@' nao reconhecido
[ERRO LEXICO] Linha 7, Coluna 9: string nao fechada -- falta '"' de
fechamento
[ERRO LEXICO] Linha 8, Coluna 7: simbolo '$' nao reconhecido

=====
Analise interrompida por erros lexicos.
=====
```

Note que mesmo com 3 erros, o lexer produziu 9 tokens validos antes de ser interrompido pela fase de controle do main.py. A análise só para ao final da tokenização, não no momento do erro.

5.3 Erros Sintáticos (Fase 2)

O parser descendente recursivo LL(1) detecta erros quando o token atual não corresponde ao esperado pela gramática. A recuperação é feita pelo método `_consumir()`: em caso de mismatch, o erro é registrado sem avançar o ponteiro, permitindo que o parser tente continuar.

1) Tipos de Erros Sintáticos

Tipo de Erro	Causa	Mensagem Gerada
Token inesperado	Token atual não é o esperado pela produção	esperado 'FIM', encontrado 'se' (SE)
Comando invalido	Token não inicia nenhum comando reconhecido	comando invalido: 'faca' (FACA)
Expressão invalida	Token não inicia uma expressão válida	expressao invalida: 'fim' (FIM)
Declaração invalida	Token no nível raiz não inicia declaração	declaracao invalida: 'fim' (FIM)
ID sem operador	Identificador não seguido de '=' ou '.'	esperado '=' ou '.' apos identificador 'x'

2) Implementação da Recuperação Sintática

```
# Em parser.py -- _consumir() nao avanca em caso de erro:
def _consumir(self, tipo: TipoToken) -> Token:
    tok = self._atual()
    if tok.tipo == tipo:
        return self._avancar() # consumo normal
    self.erros.append(ErroSintatico(
        f"esperado '{tipo.name}', encontrado '{tok.valor}'",
        tok.linha, tok.coluna
    ))
```

```
return tok    # retorna sem avançar -- recuperacao por continuidade
```

3) Saida Real com 2 Erros Sintáticos

```
$ python main.py examples/erro_sintatico.pks

[ FASE 1 ]  Analise Lexica
OK  --  18 tokens gerados

[ FASE 2 ]  Analise Sintatica
-----
Erros sintaticos encontrados (2):
[ERRO SINTATICO] Linha 9, Coluna 1: esperado 'FIM', encontrado 'se' (SE)
[ERRO SINTATICO] Linha 10, Coluna 5: esperado 'ENTAO', encontrado 'mostrar'
(MOSTRAR)

=====
Analise interrompida por erros sintaticos.
=====
```

O primeiro erro ocorre porque o bloco 'pokemon dragonite' não tem 'fim'. O parser esperava 'FIM' para encerrar o bloco, mas encontrou 'se'. O segundo erro ocorre no 'se' que está faltando 'então'. Ambos os erros são detectados na mesma execução.

5.4 Erros de Execução (Fase 3)

Erros semânticos só podem ser detectados em tempo de execução, pois dependem dos valores atribuídos as variáveis. Eles são representados pela exceção `ErroExecucao` e interrompem a execução quando ocorrem.

1) Tipos de Erros de Execução

Tipo de Erro	Quando Ocorre	Exemplo
Variável não declarada	Uso de variável sem atribuição previa	`mostrar resultado` (resultado nunca definido)
Pokémon não declarado	Bloco batalha com Pokémon inexistente	`batalha x vs y faca` (x ou y nao declarados)
Atributo inexistente	Acesso a atributo não definido no Pokémon	`dragonite.velocidade` (velocidade nao existe)
Divisão por zero	Divisor da operação '/' e zero	<code>resultado = 10 / 0</code>
Tipo incompatível	Coerção invalida para inteiro	<code>inteiro x = "texto"</code>
Loop infinito (proteção)	Laco enquanto executa mais de 10.000 iterações	Condição nunca se torna falsa

2) Implementacao da Classe `ErroExecucao`

```
class ErroExecucao(Exception):
    def __init__(self, mensagem, linha=None):
        self.mensagem = mensagem
        self.linha = linha
        super().__init__(mensagem)

    def __repr__(self):
        loc = f" (linha {self.linha})" if self.linha else ""
        return f"[ERRO EXECUCAO]{loc}: {self.mensagem}"
```

3) Saída Real com Erro de Execução

```
$ python main.py examples/erro_execucao.pks

[ FASE 1 ] OK -- 20 tokens gerados
[ FASE 2 ] OK -- AST construida

[ FASE 3 ] Execucao
-----

[ERRO EXECUCAO] (linha 10): variavel 'multiplicador' nao foi
declarada

Estado final das variaveis:
[pokemon] dragonite:
    hp      = 100
    ataque  = 80

Tokens      : 20
Erros lex   : 0
Erros sin   : 0
Status      : ACEITO E EXECUTADO
```

5.5 Tabela Consolidada de Todos os Erros

A tabela a seguir consolida todos os erros que o compilador PokeScript pode detectar, organizados por fase e tipo:

#	Fase	Código	Descrição do Erro
L1	Léxica	Char invalido	Caractere não pertence ao alfabeto da linguagem
L2	Léxica	String não fechada	Literal string sem aspas de fechamento até o fim da linha
S1	Sintática	Token inesperado	Token atual difere do esperado pela produção gramatical
S2	Sintática	Comando invalido	Nenhuma produção aceita o token atual como início de comando

S3	Sintática	Expressão invalida	Nenhuma produção aceita o token atual como início de expressão
S4	Sintática	Decl. invalida	Token no nível superior não inicia nenhuma declaração
S5	Sintática	ID sem operador	Identificador não e seguido de '=' ou '.' como esperado
E1	Execução	Var. não declarada	Variável referenciada antes de qualquer atribuição
E2	Execução	Pokémon não declarado	Nome usado em batalha/acesso não foi declarado como pokémon
E3	Execução	Atrib. inexistente	Atributo acessado não foi definido no bloco pokémon
E4	Execução	Divisão por zero	Operando direito da divisão e zero em tempo de execução
E5	Execução	Tipo incompatível	Valor não pode ser convertido para o tipo declarado
E6	Execução	Loop infinito	Laco enquanto excedeu 10.000 iterações sem terminar
E7	Execução	No desconhecido	Tipo de no da AST não tem método de execução mapeado

5.6 Organização do Projeto

O projeto está organizado em um único diretório pokescript/, com separação clara de responsabilidades entre os módulos:

```
pokescript/
  lexer.py           # Fase 1 -- Analisador Lexico
  ast_nodes.py       # Nos da AST + ImpressorAST
  parser.py          # Fase 2 -- Analisador Sintatico (LL(1))
  interpreter.py     # Fase 3 -- Interpretador tree-walker
  main.py            # Ponto de entrada (orquestra as 3 fases)
  README.md          # Documentacao e instrucoes de uso
  examples/
    valido.pks       # Programa valido completo
    batalha.pks      # Demonstracao do bloco batalha
    erro_lexico.pks   # 3 erros lexicos
    erro_sintatico.pks # 2 erros sintaticos
    erro_execucao.pks # 1 erro de execucao
```

1) Módulos e Responsabilidades

Modulo	Classe Principal	Responsabilidade
<code>lexer.py</code>	<code>Lexer</code>	Tokenização, classificação de tokens, recuperação léxica

